



DEPARTMENT OF COMPUTER AND COMMUNICATION SYSTEMS ENGINEERING
FACULTY OF ENGINEERING
UNIVERSITY PUTRA MALAYSIA
43400 UPM SERDANG
SELANGOR

COURSE	ECC4948 - COMPUTER AND COMMUNICATION SYSTEMS DESIGN PROJECT
SEMESTER	2 (2023/2024)
COORDINATOR	DR. MUHAMMAD SHAUFIL ADHA BIN SHAWKANY HAZIM
SUPERVISOR	PROF. MADYA DR. MUHAMMAD HAFIZ B. ABU BAKAR
GROUP	5
DATE	7 July 2024

FINAL REPORT: STOCKWISE - SMART SHELF

NAME	MATRIC NUMBER
AMIRA SYARINA BINTI MORMAT	210086
NUR AMELLIA BINTI MOHD FADILL	211510
ASLAM OSAMA HUSAINY BIN ABDUL HANIFF	212745
NIK MUHAMMAD SYAZWAN BIN SAMSUDDIN	212213
MUHAMAD ZULHISYAM BIN MAT SANI	209997

TABLE OF CONTENTS

1.0 EXECUTIVE SUMMARY.....	2
2.0 PROBLEM STATEMENT.....	3
3.0 PROJECT OBJECTIVES.....	4
4.0 SCOPE AND LIMITATION OF THE PROJECT.....	4
4.1 PROJECT SCOPE.....	4
4.2 PROJECT LIMITATION.....	5
5.0 SOLUTION AND SIGNIFICANCE OF PROJECT.....	6
5.1 SOLUTION OVERVIEW.....	6
5.2 SIGNIFICANCE OF PROJECT.....	6
6.0 MARKET REVIEW & CLIENT VALIDATION SUMMARY.....	8
6.1 MARKET REVIEW.....	8
6.2 CLIENT VALIDATION SUMMARY.....	9
7.0 FINAL PRODUCT DESIGN.....	12
7.1 TECHNICALITY OF HARDWARE SETUP.....	12
7.2 TECHNICALITY OF SOFTWARE SETUP.....	14
7.3 PROTOTYPE DESIGN & FUNCTIONALITIES.....	16
8.0 RESULTS.....	19
8.1 REAL-TIME INVENTORY TRACKING.....	20
8.2 AUTOMATED ALERT.....	21
8.3 DATA ANALYTICS.....	21
8.4 LIQUID CRYSTAL DISPLAY (LCD).....	22
9.0 PROJECT COSTING & POTENTIAL PROFIT.....	24
9.1 PROJECT COSTING.....	24
9.2 POTENTIAL PROFIT.....	24
10.0 RELATION TO INDUSTRIAL REVOLUTION (IR) 5.0.....	26
11.0 INDIVIDUAL CONTRIBUTION.....	28
12.0 REFERENCES.....	29
13.0 APPENDICES.....	30

1.0 EXECUTIVE SUMMARY

The StockWise: Smart Shelf project is an innovative solution designed to transform inventory management in the retail industry by leveraging cutting-edge Internet of Things (IoT) technology. The project addresses significant issues such as inefficiency and inaccuracy in traditional inventory management systems. The primary goal is to develop a smart shelf system that offers real-time data tracking, automates inventory updates, and provides comprehensive data analytics to aid retailers in making informed decisions.

Key Features of StockWise

- **Real-Time Inventory Tracking:** The StockWise system integrates Radio Frequency Identification (RFID) technology to monitor stock levels continuously on shelves. This allows for immediate detection of changes in inventory, thereby reducing the likelihood of stockouts and overstock situations.
- **Automated Alerts:** The system includes an automated alert feature that sends email notifications to store personnel when stock levels fall below a predetermined threshold, ensuring timely restocking.
- **Data Analytics:** The system provides actionable insights into inventory trends and patterns, assisting retailers in optimising stock levels and improving operational efficiency.
- **User-Friendly Interface:** The software component, developed using Visual Studio Code (VS Code), Python, and Streamlit, offers a user-friendly interface that enables store managers to interact seamlessly with the system.
- **Dynamic LCD:** Displays product information, real-time pricing updates, stock levels and highlights best-selling items to enhance the shopping experience for customers.

The final product consists of a fully functional smart shelf prototype, a software interface for easy management, and a comprehensive data analytics module. The project involved designing, developing, and testing a functional prototype, collaborating with local retailers such as BHPetrol Stations, MR.DIY, and Coopmart UPM for feedback. Despite budget and resource constraints, which limited the inclusion of some advanced features, the system's comprehensive testing in real-world environments has demonstrated its effectiveness and reliability.

StockWise sets a new benchmark in retail inventory management by enhancing operational efficiency, minimising human error, and ensuring customer satisfaction through

optimal product availability. The system's scalability and flexibility make it suitable for various retail environments, from small convenience stores to large supermarkets. By aligning with sustainable retail practices, StockWise contributes to environmental sustainability by optimising inventory levels and reducing waste. This project promises to enhance the profitability and efficiency of retail operations, offering a competitive advantage and paving the way for future advancements in inventory management technology.

2.0 PROBLEM STATEMENT

The retail industry faces significant challenges in inventory management, leading to inefficiencies, stockouts, and, eventually, lost sales and customer dissatisfaction. This is because 24% of small businesses still rely on outdated pen-and-paper methods for inventory tracking [1]. This not only causes inefficiencies and inaccuracies but also contributes to overstocking slow-moving items and understocking popular products, affecting sales and customer experience. Adding to these challenges is the fact that 74% of businesses across all industries have experienced critical parts and materials shortages [1]. These issues can significantly disrupt operations and negatively influence customer satisfaction. Furthermore, manual inventory tracking is prone to human error, leading to discrepancies between actual stock levels and recorded data, which has a significant financial impact on businesses [2]. Additionally, traditional shelves in retail stores cannot alert workers about low stock levels, leading to significant revenue loss as customers turn to alternative brands or stores [3]. The absence of real-time data and automated alerts in conventional shelving systems necessitates manual stock checks, which are inefficient and unreliable [3]. Implementing smart shelves with real-time inventory tracking can address these issues by providing automatic replenishment alerts, eliminating the need for manual stock checks and ensuring timely restocking, reducing the risks of stockouts and maximising product availability for customers.

Without inventory tracking, businesses are much more likely to encounter forecasting problems because inventory forecasting relies on informed predictions and estimations of the amount of inventory needed for a specific timeframe [4]. Without accurate data on current stock levels and past sales trends, these forecasts become unreliable. This can lead to both stockouts and overstocking, thereby reducing customer satisfaction. Investing in automated inventory management systems and data analytics tools can mitigate manual errors and optimise stock levels. According to a Bain and Microsoft survey, 56% of businesses aim to increase investment in demand forecasting and predictive planning [5]. This strategic shift toward data-driven

decision-making emphasises the importance of accurate inventory data. These statistics highlight the significant impact of manual stock checks on inventory management, resulting in frequent stockouts, overstocking, and inventory shrinkage due to human error. It underscores the importance of accurate, timely, and reliable data in optimising inventories and adapting to supply chain disruptions.

3.0 PROJECT OBJECTIVES

1. To enhance inventory management with real-time updates accessible via software on retail store devices
2. To implement automated alerts for real-time tracking of inventory levels to minimise stockouts and overstocks, optimising stock levels and preventing lost sales.
3. To enhance the shopping experience by utilising RFID tags and Liquid Crystal Display (LCD) to provide customers with real-time pricing updates and highlight best-selling items.
4. To utilise a data analytics system based on historical sales trends, further enhancing forecasting accuracy to anticipate future demand and optimise purchasing decisions.

4.0 SCOPE AND LIMITATION OF THE PROJECT

4.1 PROJECT SCOPE

The scope of the StockWise project involves designing, developing, implementing, and validating a smart shelf system tailored to improve inventory management in retail environments. The project addresses common challenges retailers face, such as inaccurate stock levels, delayed inventory updates, and inefficient restocking processes. By leveraging RFID technology, the system enables real-time inventory tracking and uses data analytics to provide valuable insights for inventory optimisation. Key activities within the project scope include:

- Designing a smart shelf prototype integrating RFID tags and readers to monitor inventory.
- Developing a user-friendly software interface using VS Code, Python, and Streamlit for easy interaction with the system.
- Implementing data analytics to process and analyse inventory data, providing retailers with actionable insights.

- Conducting comprehensive testing and validation of the system in real-world retail environments to ensure its effectiveness, and reliability and meets industry standards and client requirements.
- Integrating automated email alerts to notify store personnel when stock levels fall below a predetermined threshold.
- Collaborating with local retailers such as BHPetrol Stations, MR.DIY and Coopmart UPM to gather feedback and refine the system based on their specific needs.

The project is focused on delivering a functional prototype demonstrating the StockWise system's effectiveness in improving inventory management. The long-term goal is to develop a scalable and adaptable solution that can be integrated into various retail settings.

4.2 PROJECT LIMITATION

Despite the innovative approach and potential benefits of the StockWise system, the project has certain limitations. One primary constraint is that the current project phase only covers the development and testing of a prototype. This limits the comprehensive evaluation of the system's long-term reliability and scalability across various retail settings. Another significant limitation is the exclusion of certain advanced features due to budget constraints. Initially, the project intended to include an automatic push system and predictive analytics; however, these features were replaced with basic data analytics due to resource limitations. Additionally, the system does not currently offer maintenance services, which are essential for ensuring ongoing functionality and performance.

The development phase encountered challenges with the initially planned software, Odoo Community Edition. The complexity and steep learning curve of Odoo Community Edition made it challenging to use effectively. Therefore, the project transitioned to a more user-friendly and manageable software setup using VS Code, Python, and Streamlit. This change facilitated the development of a more intuitive and accessible interface for the StockWise system. However, it also meant that some features initially planned for Odoo could not be implemented within the new software framework due to time and resource constraints.

The project also does not fully explore the integration of the smart shelf system with a wide range of existing retail management software. This could potentially limit its adaptability to different retail environments. Furthermore, the selection of materials and

technologies was influenced by budget constraints, which might impact the overall reliability and durability of the system.

5.0 SOLUTION AND SIGNIFICANCE OF PROJECT

5.1 SOLUTION OVERVIEW

StockWise is an advanced smart shelf system designed to revolutionise inventory management in the retail sector. The solution integrates cutting-edge technologies such as IoT, artificial intelligence (AI), and real-time data analytics to provide a comprehensive and automated inventory management system. Key features of StockWise include real-time inventory tracking, automated alerts, data analytics, and dynamic LCDs.

1. **Real-Time Inventory Tracking:** This system uses sensors and RFID technology to provide accurate, real-time data on inventory levels, reducing the risk of stockouts and overstocking.
2. **Automated Alerts:** This system sends email alerts for low stock levels to prevent stockouts, help maintain optimal inventory levels, and ensure timely restocking.
3. **Data Analytics:** This feature uses past sales data to analyse trends and patterns, helping retailers make informed stocking decisions. It offers detailed analytics and reporting capabilities, enabling retailers to make decisions regarding inventory management, ordering, and sales strategies.
4. **Dynamic LCD:** These displays show product names, prices, highlight hot-selling items, and stock levels. Based on sales data, information is updated in real-time, providing customers with current and relevant product information.

5.2 SIGNIFICANCE OF PROJECT

Implementing StockWise offers numerous significant benefits to retailers, employees, and customers, contributing to the overall efficiency and profitability of retail operations.

1. **Enhanced Inventory Management:** StockWise's real-time inventory tracking significantly reduces the risk of stockouts and overstocking. Retailers can maintain optimal inventory levels, minimising holding costs and maximising sales opportunities. The system's automated alerts ensure timely restocking, leading to better inventory control and reduced waste.
2. **Improved Operational Efficiency:** Automating inventory-related tasks allows retail employees to focus on customer service and other value-added activities.

This shift enhances employee productivity and improves customers' overall shopping experience. Reducing manual labour and associated costs translates into significant operational savings for retailers.

3. **Superior Customer Experience:** StockWise enhances customers' shopping experience by ensuring that products are always available and easily accessible. Dynamic LCDs provide up-to-date information on product names, prices, and stock levels, further improving the shopping experience. A well-stocked and organised store leads to higher customer satisfaction and loyalty, ultimately driving repeat business.
4. **Data-Driven Decision Making:** StockWise's detailed analytics and reporting capabilities provide retailers with valuable insights into sales trends. By utilising past sales data to analyse trends and patterns, StockWise helps retailers make informed stocking decisions. This data enables retailers to make informed decisions about ordering, promotions, and merchandising strategies, improving sales performance and profitability.
5. **Competitive Advantage:** Adopting StockWise positions retailers at the forefront of technology adoption in the retail sector. The innovative features and efficiencies offered by the smart shelf system provide a significant competitive advantage, attracting tech-savvy customers and differentiating the retailer from competitors.
6. **Scalability and Flexibility:** StockWise is designed to be scalable and flexible, making it suitable for various retail environments, from small convenience stores to supermarkets. Its modular design allows for easy customisation and expansion, ensuring that the system can grow with the retailer's needs.
7. **Sustainability:** StockWise contributes to more sustainable retail practices by optimising inventory levels and reducing waste. The system's ability to prevent overstocking and reduce spoilage aligns with environmental sustainability goals, making it an attractive option for retailers committed to green initiatives.

In conclusion, StockWise represents a significant advancement in smart shelf technology, offering a sturdy solution to common retail challenges. Its implementation increases inventory management and operational efficiency, significantly elevates the customer experience, and provides a competitive edge. StockWise's scalability, flexibility, and sustainability further underscore its value, making it a critical investment for forward-thinking retailers.

6.0 MARKET REVIEW & CLIENT VALIDATION SUMMARY

6.1 MARKET REVIEW

The adoption of smart shelf technology in the retail sector has been gaining significant traction, driven by the need for enhanced inventory management, operational efficiency, and improved customer experience. StockWise aims to capitalise on this growing trend and carve out a significant market share. The target audience for StockWise includes retailers such as supermarkets, department stores, speciality stores, and convenience stores that require efficient inventory management systems. Retail employees who manage inventory and provide customer service will benefit from automated time-tracking solutions that reduce manual counting stock tasks. Customers seeking a seamless shopping experience with well-stocked and organised shelves are also a crucial segment. Business owners and executives focused on improving operational efficiency and customer satisfaction; investors and stakeholders looking to invest in innovative retail technologies represent additional key target groups.

The smart shelves market is experiencing robust growth, valued at USD 3.0 billion in 2022 and projected to reach USD 8.3 billion by 2027, with a Compound Annual Growth Rate (CAGR) of 22.4% [6]. This rapid growth is driven by increasing retail sector automation and related technological advancements. Europe currently leads the market due to its advanced retail infrastructure and high technology adoption rates. At the same time, the Asia-Pacific region is expected to grow at the highest CAGR, spurred by the rapid development of the retail sector and increasing investment in smart retail solutions [6].

Several factors drive the adoption of smart shelves. One such factor is enhanced inventory management capabilities, which provide real-time monitoring to reduce the risk of stockouts and overstocking, thus leading to cost savings and improved stock accuracy. Operational efficiency serves as another key driver, as automation decreases labour costs and permits staff to concentrate on customer service and other value-added activities. Furthermore, customer experience is significantly improved through well-stocked shelves and timely restocking, which increases satisfaction and loyalty.

The smart shelves market is competitive, with key players including SES-imagotag, Pricer, Trax, SOLUM, and Avery Dennison. These companies focus on innovation, strategic partnerships, and product portfolio expansion. Opportunities for StockWise are

abundant, particularly in emerging markets like Asia-Pacific and Latin America, where the retail sectors are rapidly growing. Technological advancements, such as integrating AI, IoT, and machine learning, can further enhance the capabilities of smart shelves, opening up new applications and improving performance. However, challenges remain, including the high initial costs associated with smart shelf technology, which can be a barrier for smaller retailers. Demonstrating a clear return on investment (ROI) is crucial for overcoming this challenge. Market awareness is also essential, as educating potential customers about the benefits and ROI of smart shelves is key to driving adoption.

In conclusion, StockWise is well-positioned to capitalise on the growing demand for smart shelf technology in the retail sector. By targeting key market segments and leveraging its unique features, StockWise can effectively compete with established players and capture a significant share of the market. The combination of market growth, technological advancements, and strategic positioning offers a promising outlook for the adoption and success of StockWise smart shelves.

6.2 CLIENT VALIDATION SUMMARY

Client validation was a critical component of the StockWise project to ensure that the final product met the needs and expectations of the target users. Thorough interviews and feedback sessions were conducted with stakeholders from Selama Pelangi (M) Sdn Bhd (BHPetrol), MR.DIY, and Coopmart UPM. These interactions provided valuable insights into the specific challenges faced by retailers in managing their inventory.

i. Feedback from Selama Pelangi (M) Sdn Bhd (BHPetrol)

BHPetrol highlighted the challenges of managing inventory across multiple shelf locations with limited staff. They emphasised the need for real-time tracking to prevent stockouts and overstocking, which were common issues due to manual inventory checks.

ii. Feedback from MR.DIY

MR.DIY pointed out the inefficiencies and inaccuracies of their current manual inventory management processes. They required a system capable of reducing the manual workload significantly through features like LCD to display the stock levels. While detailed analytics were not explicitly mentioned, they are required for MR.DIY to provide insights into inventory trends and support better

decision-making. Automated alerts for low stock levels were another critical feature to ensure timely restocking and avoid product shortages.

iii. **Feedback from Coopmart UPM**

Coopmart UPM's feedback centred on the need for a cost-effective solution that could be easily implemented and scaled as their business grew. They needed a system that offered both real-time inventory tracking and comprehensive data analytics to improve stock management. Coopmart UPM also highlighted the importance of a reliable alert system to notify staff of low stock levels, reducing the risk of stockouts and enhancing customer satisfaction.

From these discussions, it became clear that real-time inventory tracking, a user-friendly interface, and seamless integration with existing systems were essential. Clients expressed frustration with current inventory management practices, highlighting issues such as stockouts, overstocking, and the time-consuming nature of manual inventory checks. They also emphasised the need for a system that could provide real-time alerts and detailed analytics to support better decision-making.

Incorporation of Feedback into the Final Product

The feedback gathered was crucial in shaping the design and functionality of the StockWise system.

- **Real-Time Inventory Tracking:** The integration of RFID technology was prioritised to ensure accurate and real-time tracking of inventory levels, addressing the common issue of stock inaccuracies highlighted by all clients.
- **User-Friendly Interface:** The software interface developed using Streamlit focused on ease of use and accessibility. This ensured that store managers could quickly and efficiently interact with the system.
- **Seamless Integration:** The system was designed to integrate with existing inventory management software, aligning with BHPetrol's need for streamlined operations.
- **Detailed Data Analytics:** The data analytics component was designed to provide detailed insights into inventory trends, helping retailers to optimise their stock levels. This addressed the needs of both MR.DIY and Coopmart UPM for better decision-making tools.

- **Automated Email Alerts:** The inclusion of automated email alerts ensured that store personnel received timely notifications about low stock levels, enabling prompt restocking and minimising the risk of stockouts, as requested by all clients.

Feedback on the StockWise Prototype

After presenting the StockWise prototype to the clients, we received specific feedback on its performance:

i. Selama Pelangi (M) Sdn Bhd (BHPetrol)

BHPetrol appreciated the real-time tracking capability and found the integration of the inventory system smooth and effective. They were particularly impressed with the accuracy and timeliness of the inventory updates, which helped in reducing stockouts and overstocking issues. The seamless integration made the transition to using StockWise efficient and hassle-free.

ii. MR.DIY

MR.DIY found the automated alerts for low stock levels particularly useful and acknowledged the potential of the system to significantly reduce manual inventory checks. They highlighted the user-friendly interface and the clear, immediate notifications as key strengths. The ability to get detailed insights into inventory trends and the ease of generating reports were also highly appreciated, which they believe will support better decision-making.

iii. Coopmart UPM

Coopmart UPM was impressed with the cost-effectiveness and scalability of the solution. They found the real-time tracking and data analytics features to be highly beneficial for their operations. The system's ability to provide comprehensive data analytics helped them better understand their stock levels and trends, facilitating more informed and strategic decisions. They also appreciated the automated email alerts, which ensured timely restocking and maintained optimal product availability.

Overall, the client validation process confirmed that the StockWise system addresses clients' critical pain points, offering a reliable solution that enhances operational efficiency and customer satisfaction. The final product is a practical, user-friendly, and integrative smart shelf system tailored to the specific needs of the retail industry.

7.0 FINAL PRODUCT DESIGN

7.1 TECHNICALITY OF HARDWARE SETUP

The hardware setup of the StockWise: Smart Shelf system is designed to ensure precise, real-time inventory management. The key components include:

1. Arduino RFID Tags and Readers (RC522)

Each product is equipped with an RFID tag containing a unique identifier. These tags communicate with RFID readers installed on the StockWise system. The readers are responsible for continuously scanning the RFID tags and transmitting data regarding the presence and quantity of items on the shelf. This setup allows for accurate, real-time inventory tracking without the need for manual intervention.

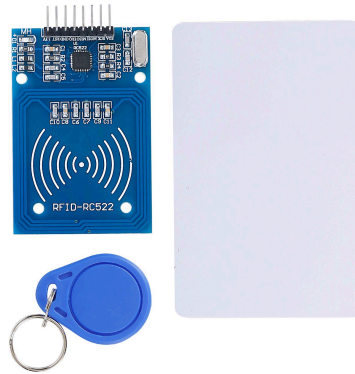


Figure 1: The RFID tags & readers.

2. Arduino Uno R3 Microcontroller

The Arduino Uno R3 microcontroller is a central component of the system, responsible for interfacing with the RFID readers and managing the data flow. Arduino is chosen for its versatility and ease of use. The microcontroller processes the data from the RFID readers and sends it to the central processing unit (CPU) for further handling. This allows for flexible and scalable hardware integration. By incorporating Arduino, the StockWise system benefits from enhanced flexibility, ease of prototyping, and the ability to easily expand or modify the hardware components as needed. Furthermore, the use of Arduino simplifies the development process, allowing for quicker iterations and improvements based on real-world testing and feedback.



Figure 2: Arduino Uno R3.

3. Central Processing Unit (CPU)

The system uses an Arduino microcontroller to interface with the RFID readers. The Arduino collects data from the RFID readers and processes it before sending it to the CPU, which is the core component responsible for processing and managing the inventory data. The CPU processes the data received from the Arduino, ensuring that it is accurately recorded and updated in the system, reflecting real-time stock levels.

4. Liquid Crystal Display (LCD)

The system includes LCD screens attached to the smart shelves. These displays provide visual indicators of product information and inventory status, showing real-time updates on stock levels. They alert store personnel to low stock levels or other issues that require immediate attention, facilitating timely restocking and efficient inventory management.



Figure 3: The LCD.

5. Network Connectivity

The hardware components are connected through a network that enables seamless communication between the RFID readers, the CPU, and the LCDs. This

network ensures that data flows efficiently and is updated in real-time across all system components.

7.2 TECHNICALITY OF SOFTWARE SETUP

The StockWise system leverages modern development tools and frameworks. This approach ensures a seamless, user-friendly experience while also providing reliable data processing and analytics for efficient inventory management solutions. The key aspects of the software setup include:

1. Visual Studio Code (VS Code)

The StockWise system software is developed using VS Code, an open-source code editor that supports various programming languages and development tools. It offers a wide range of extensions and tools that streamline the development process. VS Code provides a streamlined environment for writing, debugging, and deploying software. VS Code's debugging capabilities and integrated Git support make it an ideal choice for developing the StockWise software in Python. This setup provides an environment where developers can write, debug, and deploy Python scripts efficiently. Python scripts handle data collection, processing, and analysis.

2. Streamlit Documentation under Application Programming Interface (API) References

The Streamlit framework simplifies the creation of an intuitive and interactive web-based user interface for data-driven applications. It allows rapid development, deployment, and building of visually appealing dashboards with minimal coding effort. Streamlit's real-time updating capabilities are crucial for displaying inventory data dynamically. This enables store managers to access and interact with the StockWise system through a web browser, ensuring platform independence and accessibility.

3. Arduino Integrated Development Environment (IDE)

The Arduino IDE is used to write and upload code to the Arduino microcontrollers, which are integral to the StockWise system. The Arduino IDE supports the C and C++ programming languages using special rules of code structuring. The code written in the Arduino IDE is uploaded to the Arduino microcontrollers via a USB connection. These microcontrollers handle various tasks within the StockWise

system, such as reading data from the RFID tags attached to each item, processing this data to track item movement and status, and communicating with the CPU. This enables real-time inventory management and tracking within StockWise. The use of the Arduino IDE simplifies the process of programming the microcontrollers, ensuring that they function correctly as part of the larger StockWise system.

4. User Interface

The user interface, developed using Streamlit, is designed to be intuitive and user-friendly for store managers. It allows store managers to monitor inventory levels, generate graphs, and access data analytics with ease. The interface includes features such as real-time inventory updates, data visualisation, and automated alerts for low-stock situations. It is designed to be easy to navigate, with a focus on usability and accessibility. Streamlit's ability to update in real-time ensures that users always have the latest information at their fingertips.

5. Data Collection and Processing

The data collected from the RFID readers is processed by the Arduino microcontroller and sent to the CPU. Python scripts running on the CPU handle the data collection, ensuring that it is accurately recorded and updated in the system. These scripts are responsible for managing the database, performing data validation, and ensuring the integrity of the inventory data.

6. Database Management

The system's software includes modules for managing the inventory data collected by the RFID readers. These modules process the raw data, update the inventory status in real-time, and store the data in a structured format. The software ensures data integrity and provides reliable mechanisms for data retrieval and analysis. A scalable database system efficiently handles large volumes of data, enabling quick access and retrieval. The data structure is optimised for inventory management, with tables and relationships structured to support fast queries and updates. Python's seamless integration with database management systems (such as SQLite, MySQL, or PostgreSQL) ensures smooth data handling.

7. Data Analytics

The data analytics module processes the inventory data to provide valuable insights into inventory trends and patterns. This module provides retailers with actionable insights to optimise stock levels, improve order management, and enhance overall operational efficiency. The analytics component is designed to be scalable, allowing for the integration of more advanced features, such as predictive analytics, in future iterations. Python libraries such as Pandas and NumPy are used for data manipulation and analysis, while visualisation libraries like Matplotlib and Seaborn create informative charts and graphs. These analytics help retailers make informed decisions regarding stock levels, product placement, and order management strategies.

8. Automated Alerts

The system includes an automated alert feature that sends email notifications to store personnel when stock levels fall below a predetermined threshold. The email alerts are configured to provide relevant details, such as the product name, current stock level, and recommended restocking actions. This ensures timely restocking and minimises the risk of stockouts. This feature is implemented using Python's email libraries (such as smtplib) to ensure timely notifications.

7.3 PROTOTYPE DESIGN & FUNCTIONALITIES

The StockWise prototype's design centres on demonstrating the smart shelf system's core functionalities and potential benefits. The prototype integrates RFID technology to track item removal in real-time. The central unit processes this data and displays it on an LCD screen, providing instant updates on inventory levels. Key functionalities of the prototype include:

- **Automated Inventory Tracking**

The RFID tags and readers work together to provide continuous, real-time updates on inventory status. This automation eliminates the need for manual stock checks, reducing labour costs and minimising human error. The data collected by the RFID readers is synchronised with the central database in real-time, ensuring that inventory records are always up-to-date.

- **Real-Time Alerts**

The system can generate real-time alerts for low stock levels, enabling store personnel to restock shelves promptly and ensure that products are always available for customers. Automated email alerts further enhance this functionality by notifying staff when stock levels fall below a predetermined threshold, ensuring timely restocking.

- **Data Analytics**

The prototype includes a basic data analytics module that processes inventory data to identify trends and patterns. This information helps retailers make informed decisions about stock levels, product placement, and order management. The analytics module uses visualisation libraries to create informative charts and graphs, making it easier for users to interpret the data.

- **User-Friendly Interface**

Developed using Streamlit, the interface allows store managers to interact with the system easily, access inventory data, and generate reports. It is designed to be intuitive, ensuring that users can quickly learn how to operate the system without extensive training.

The StockWise prototype, as shown in Figure 4, was tested during a visit to SMK Agama Maahad Hamidiah, Kajang. This testing ensured that the system meets practical needs and performs reliably under real-world conditions. Feedback from these tests has been used to refine the system, address any issues, and enhance its functionality. Overall, the StockWise prototype represents a significant advancement in retail technology, offering a comprehensive solution to the challenges of traditional inventory management. By providing real-time tracking, automated updates, and actionable insights, the StockWise system has the potential to significantly improve the efficiency and effectiveness of retail operations.

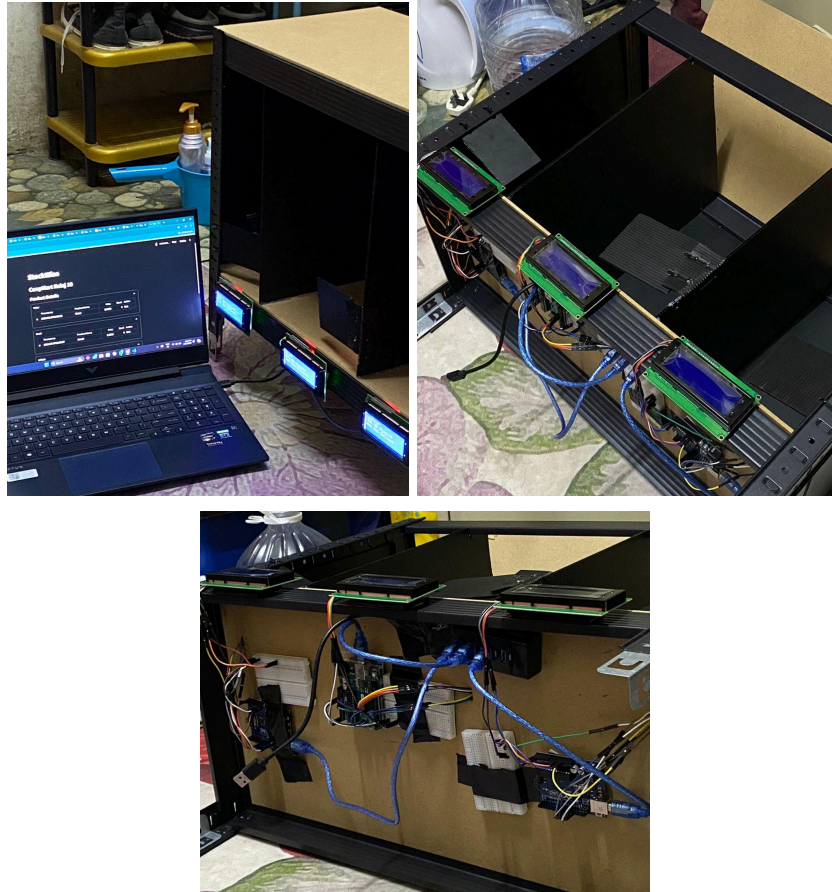


Figure 4: StockWise prototype.

A block diagram is included below to illustrate the design and functionality of the StockWise system further. This diagram provides a visual representation of the key components and their interactions:

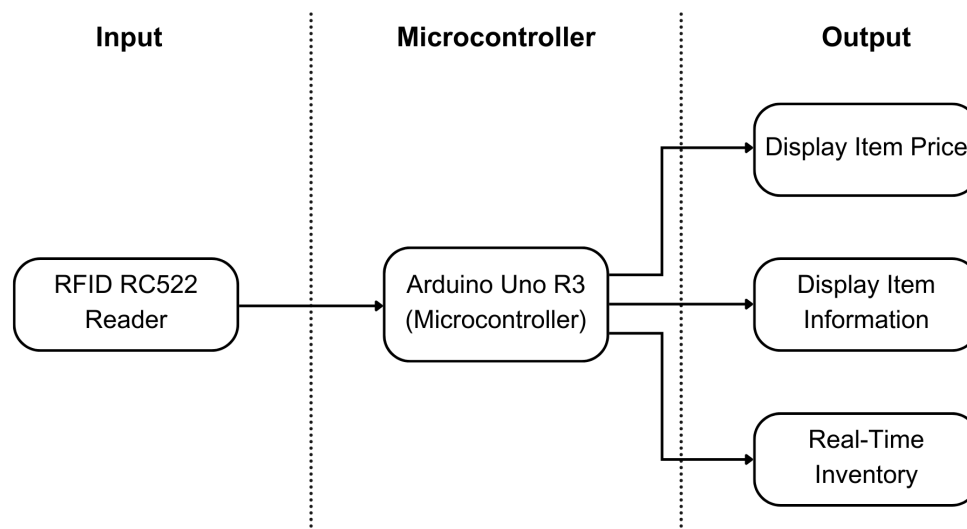


Figure 5: StockWise block diagram

StockWise will detect any item that is removed from the shelves using RFID tags attached to each product. The RFID RC522 reader will read the radio frequencies of the items, and the LCD screen will display essential information such as the product names, prices, current stock levels, and whether the items are hot-selling. This real-time display ensures that store personnel have immediate access to essential inventory data. The Arduino Uno R3 will serve as the CPU, integrating and managing all system modules to ensure seamless operation in accordance with Industry 5.0 principles. This includes the capture, processing, and display of inventory data, as well as communication with the software component that records all items moving in and out of the shelves.

Retail store workers play a vital role in this system by maintaining the shelves and ensuring the accuracy of stock quantities through the software. This collaboration ensures that the digital inventory records are always up-to-date and match the physical stock on the shelves. This interaction highlights the project's theme of human-robot collaboration and the integration of machine learning within retail operations.

By leveraging RFID technology, real-time data display on LCD screens, and comprehensive inventory management software, StockWise significantly enhances the efficiency and accuracy of inventory management in retail stores. The system automates routine tasks, reducing manual labour and human error, while still supporting essential human oversight and intervention. This creates a balanced synergy between technological automation and human labour, embodying the essence of Industry 5.0, where advanced technologies work alongside humans to enhance productivity and operational efficiency.

8.0 RESULTS

The results from the StockWise implementation demonstrate the effectiveness of real-time inventory tracking and data analytics in improving inventory management and operational efficiency. The dataset used for this analysis was collected from SMK Agama Maahad Hamidiah, Kajang, during STEM Day, providing real-world insights into the StockWise system's performance. This section presents the data collected, analyses the findings, and discusses the implications for retail operations.

8.1 REAL-TIME INVENTORY TRACKING

Product Details					
Water					
	Timestamp	Product Name	Price	Stock	Action
0	2024-06-27 10:01:36	Water	2.0000	4	Out
Snack					
	Timestamp	Product Name	Price	Stock	Action
0	2024-06-27 10:04:01	Snack	1.0000	2	Out
1	2024-06-27 10:02:46	Snack	1.0000	3	Out
2	2024-06-27 10:02:02	Snack	1.0000	4	Out
Lollipop					
	Timestamp	Product Name	Price	Stock	Action
0	2024-06-27 10:03:47	Lollipop	1.5000	2	Out
1	2024-06-27 10:03:37	Lollipop	1.5000	3	Out
2	2024-06-27 10:03:15	Lollipop	1.5000	4	Out

Figure 6: Real-time tracking of the stock out.

Figure 6 illustrates the real-time tracking data from StockWise. The interface, developed using Streamlit, displays detailed product information, including timestamps, product names, prices, stock levels, and actions (e.g., items being taken out). Moreover, since there are three products illustrated in the figure, it shows that we have three partitions. First partition is water, second partition is snack, and lastly, the third partition is the lollipop.

As shown in the Figure 6, StockWise accurately detects changes in inventory levels. For instance, at 10:04 A.M. on June 27, 2024, one snack was taken out, reducing the stock level to 2. This real-time update ensures precise monitoring of stock levels, which is crucial for maintaining optimal inventory and preventing stockouts.

8.2 AUTOMATED ALERT

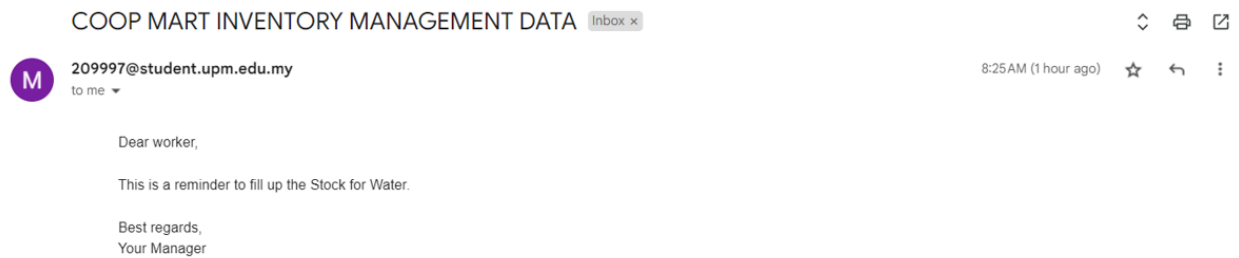


Figure 7: Email alert from StockWise.

Figure 7 demonstrates the automated alert feature of the StockWise system, showcasing an email notification sent to a worker regarding low stock levels. The email notification system is a crucial component of StockWise. It is designed to prevent stockouts by promptly informing relevant personnel when stock levels are low. In this example, an email was sent to remind the worker to restock water, ensuring that inventory levels are maintained.

8.3 DATA ANALYTICS

StockWise utilises past sales data to analyse inventory trends and patterns. By doing so, it provides valuable insights into product demand and helps in making informed stocking decisions.

Today's Sales Summary			
	Product Name	Total Sell	Total Price
0	Water	48	96.0000
1	Snack	25	25.0000
2	Lolipop	9	13.5000

Figure 8: Sales Summary of a day.

Next, Figure 8 depicts the total sales summary for the day. This comprehensive summary includes key information such as product names, total quantities sold, and the total sales price for each product. This detailed overview is invaluable for retailers, providing immediate insights into daily sales performance and aiding in strategic decision-making.

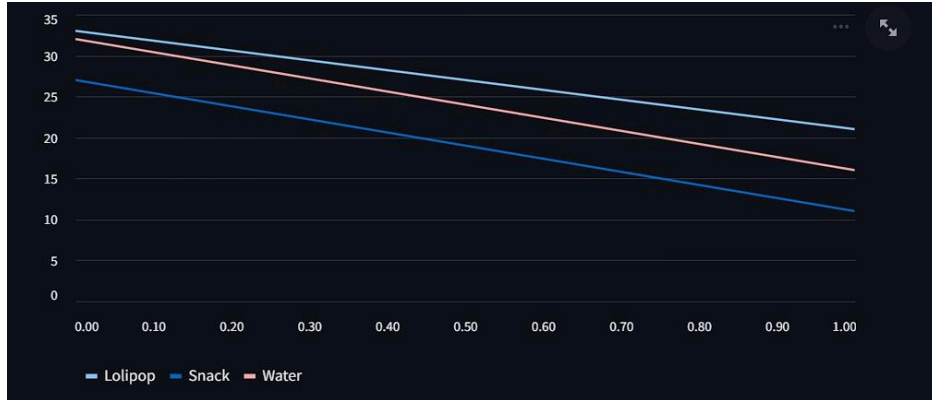


Figure 9: The graph analysis that has been created by stockwise.

Figure 9 illustrates the total restocking trend for three products: Lollipop, Snack, and Water, over a specific period. The x-axis represents time, while the y-axis indicates the quantity of products in stock. The graph portrays a clear downward trend for all three products, indicating a consistent decline in their stock levels. This decline is primarily due to the lack of restocking, as no new inventory has been added during the observed period. Lollipop: The blue line represents the stock level of lollipops. Starting at a quantity of 35, it steadily declines to around 15. Snack the pink line indicates the stock level of snacks, which starts at 30 and drops to about 10. Next is water, where the red line depicts the stock level of water, starting at 32 and declining to approximately 12. The continuous decline in stock levels for all products signifies that the existing inventory is being depleted without any replenishment.

8.4 LIQUID CRYSTAL DISPLAY (LCD)

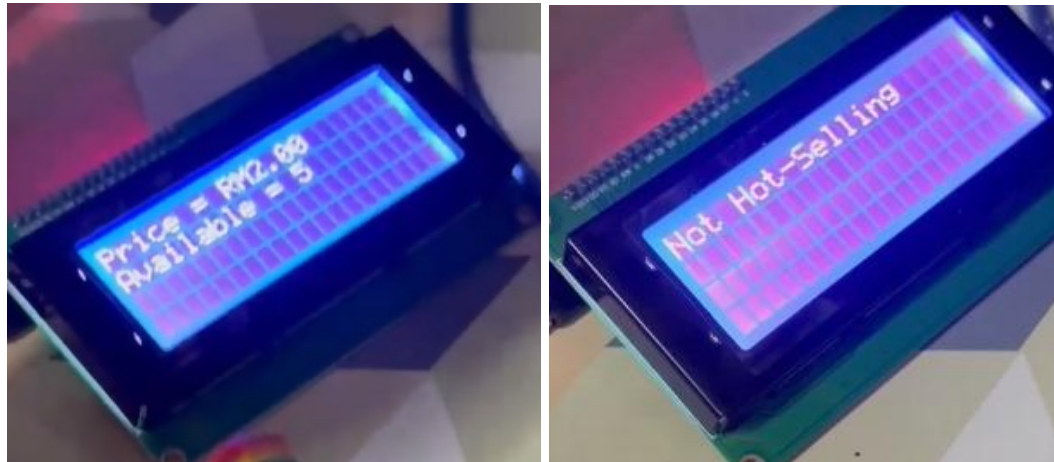


Figure 9: Price display on the LCD.

The images above showcase the functionality of the LCD screen in the StockWise smart shelf system. The display is a crucial component that provides real-time information to both retailers and customers, enhancing the overall shopping experience and streamlining inventory management. The first image displays crucial product information, including the price and the number of items available. In this case, the LCD shows "Price = RM2.00" and "Available = 3.". Therefore, it means that the product availability on the shelf is 3. Shoppers can quickly see the price and availability of products, making it easier to make purchasing decisions without needing to search for price tags or ask store employees. The second image shows the LCD indicating that a particular product is "Not Hot-Selling." This feature is part of the StockWise system's ability to classify products based on their sales performance. By identifying items that are not selling well, retailers can make informed decisions about marketing strategies, discounts, or repositioning these products within the store. This real-time feedback helps in optimizing inventory and ensuring that shelf space is utilised effectively. Retail staff can easily monitor stock levels directly on the shelf. When the availability number is low, it triggers restocking actions, verifying that popular items remain available and reducing the risk of stockouts.

9.0 PROJECT COSTING & POTENTIAL PROFIT

9.1 PROJECT COSTING

Table 1: Cost estimation of the prototype.

No	Item	Quantity	Unit Price, RM	Total price, RM
1.	Arduino Uno R3	3	0.00	0.00
2.	Shelf	1	44.29	44.29
3.	RFID Tag & Set	1	32.10	32.10
4.	Circuit Equipments	1	89.40	89.40
5.	Decoration	1	35.00	35.00
15.	Total	RM 200.79		

9.2 POTENTIAL PROFIT

Table 2: Cost estimated project of Stockwise.

No	Item	Quantity	Unit Price, RM	Total price, RM
1.	Arduino Uno R3	3	30.00	90.00
2.	Shelf	1	800.00	800.00
3.	RFID Tag & Set	1	100.00	100.00
4.	Circuit Equipments	1	89.40	89.40
5.	LCD Display	1	50.00	50.00
15.	Total	RM 1129.40		

Table 3: Labour cost for a 1 unit StockWise.

Expense	
Basic hourly wage	RM 10.00
Estimated Labour Hours per Unit	3 Hours
Total Labour cost	RM 30.00

The total estimated cost for the development and implementation of the StockWise project is cost estimated project + cost labour = RM 1129.40. To ensure a sustainable business model and cover operational expenses while generating a reasonable profit margin, we propose selling StockWise at a 20% increase over the estimated cost. This brings the proposed selling price to RM 1391.28. This pricing strategy is designed to reflect the value added by StockWise's innovative features, including real-time tracking, automated alerts, and interactive displays, which significantly enhance inventory management and sales efficiency.

Based on our proposed pricing strategy, the financial projections for selling StockWise units are as follows:

- **Total Estimated Cost per Unit:** RM 1129.40
- **Proposed Selling Price per Unit:** RM 1391.28 (20% increase)
- **Profit per Unit:** RM 261.88
 - Calculation: RM 1391.28 (selling price) - RM 1129.40 (cost)

Assuming we sell 20 units, the financial outcomes can be summarized as:

- **Total Revenue:** RM 27,825.60
 - Calculation: 20 units x RM 1391.28
- **Total Cost:** RM 22,588.00
 - Calculation: 20 units x RM 1129.40
- **Total Profit:** RM 5237.60
 - Calculation: Total Revenue - Total Cost
 - Verification: 20 units x RM 261.88 (profit per unit)

This projection illustrates the profitability of the StockWise project, highlighting a significant return on investment through our strategic pricing model. Each unit sold contributes a profit of RM 261.88, and selling 20 units would yield a total profit of RM 5237.60. The profit generated will enable further development and scaling of StockWise, ensuring its continued success in the market.

10.0 RELATION TO INDUSTRIAL REVOLUTION (IR) 5.0

StockWise strikes a balance between automation and human labour by automating routine inventory management tasks, such as tracking stock levels and sending alerts for low inventory. This automation frees up human workers to focus on more complex and value-added activities, such as customer service and strategic decision-making. By reducing the burden of manual inventory checks, StockWise enhances operational efficiency while still requiring human oversight and intervention for restocking and system maintenance. This balance exemplifies the IR 5.0 vision of human-centric automation.

The key technologies driving StockWise, in line with IR 5.0, include real-time data analytics, Internet of Things (IoT) sensors, and automated alert systems. These technologies enable continuous monitoring and accurate tracking of inventory levels, significantly improving productivity by reducing the time spent on manual stocktaking and minimising the risk of stockouts. The integration of LCD displays for real-time updates further enhances productivity by providing immediate access to crucial information, facilitating better decision-making on the shop floor. This integration of smart technologies is central to the IR 5.0 framework, which emphasises the synergy between advanced digital tools and human expertise.

StockWise contributes to the upskilling and reskilling of the workforce by introducing employees to advanced inventory management technologies and data analytics. Workers learn to interact with sophisticated systems, interpret data-driven insights, and respond to automated alerts, thereby acquiring new skills relevant to a technologically advanced retail environment. Training in these areas not only enhances their current job performance but also prepares them for future roles in an increasingly automated and data-driven industry. This focus on skill enhancement aligns with IR 5.0's goal of fostering a more capable and adaptable workforce.

Intelligent robotics and automation in StockWise enhance productivity in retail operations by ensuring optimal inventory levels and reducing the likelihood of stockouts. The system's real-time tracking and alert features streamline inventory management processes, enabling faster and more accurate restocking. This automation minimises downtime and ensures that popular items are always available, leading to higher customer satisfaction and increased sales. By optimising inventory management, StockWise supports the IR 5.0 objective of leveraging intelligent automation to boost operational efficiency and business outcomes.

StockWise supports mass customisation and personalisation in retail by providing detailed sales data and trends, which can be used to tailor inventory and marketing strategies to customer preferences. The real-time data collected by StockWise allows retailers to quickly adapt to changing consumer demands, ensuring that the right products are available at the right time. This capability enhances the customer shopping experience and allows for more personalised service. The ability to respond dynamically to consumer needs reflects IR 5.0's emphasis on creating personalised, customer-centric experiences through advanced technology.

Despite the increased automation introduced by StockWise, there is potential for a net increase in employment opportunities. The efficiency gains from automation can lead to business growth, creating new roles in system maintenance, data analysis, and customer service. Moreover, the need for human oversight and decision-making in managing automated systems ensures that there remains a demand for skilled workers. This potential for job creation is consistent with IR 5.0's vision of harmonising technological advancement with workforce development.

The implementation of StockWise and similar IR 5.0 technologies may give rise to new job profiles, such as IoT system managers, data analysts specialising in retail analytics and automation coordinators. These roles require a blend of technical knowledge and strategic thinking, reflecting the evolving demands of a technologically advanced retail environment. The emergence of these new roles highlights the transformative impact of IR 5.0 on the job market, encouraging the development of specialised skills and expertise.

StockWise aligns with the concept of Society 5.0 by integrating advanced technologies to create a more efficient, human-centred retail experience. By leveraging data analytics and automation, StockWise enhances operational efficiency and customer satisfaction while promoting a balanced relationship between technology and human labour. This alignment ensures that technological advancements contribute positively to society, fostering sustainable growth and improving quality of life. Through its innovative approach, StockWise exemplifies the principles of Society 5.0, where technology enhances human capabilities and societal well-being.

11.0 INDIVIDUAL CONTRIBUTION

Table 4: Contribution of each member.

NO	CONTRIBUTION	CONTRIBUTOR
1	Select Project Title	All
2	Create Project Timeline	Amellia
3	Create System Design	Aslam, Nik, Zul
4	Ideation Pitch	All
5	Search For Potential Clients	Amira, Amellia
6	Prepare Clients Questionnaire	Amira, Amellia
7	Interview With Potential Clients	Amira, Amellia
8	Analise Feedbacks From Clients	Amira, Amellia
9	Documentating Client Validation Report (CVR)	All
10	Documentating Project Proposal	All
11	Submit CVR And Project Proposal	Amira
12	Develop The Product	All
13	Test The Product	Aslam, Nik, Zul
14	Finalise The Product	All
15	Prepare Costing	Amellia
16	Documentation Final Report	Amira, Amellia
17	Prepare Final Presentation	All
18	Final Presentation	All
19	Submit Final Project Report	Amira

12.0 REFERENCES

- [1] “Inventory Management Statistics 2023 in Malaysia,” Adynamics, May 09, 2023. Available: <https://adynamics.com.my/inventory-management-statistics/>. [Accessed: May 29, 2024]
- [2] J. Kuhn, “The Financial Impact of Manual Inventory Record Errors,” *International Journal of Business and Social Science*, vol. 12, no. 10, 2021, doi: <https://doi.org/10.30845/ijbss.v12n10p2>. Available: https://ijbssnet.com/journals/Vol_12_No_10_October_2021/2.pdf [Accessed: May 29, 2024]
- [3] R. Satapathy, S. Prahlad, and V. Kaulgud, “Smart Shelfie -- Internet of Shelves: For Higher On-Shelf Availability,” 2015 IEEE Region 10 Symposium, pp. 70–73, May 2015, doi: 10.1109/tensymp.2015.9.
- [4] A. Belyh, “Inventory Management Statistics from Reputed Sources | 2024,” *FounderJar*, Jun. 20, 2022. Available: <https://www.founderjar.com/inventory-management-statistics/>. [Accessed: May 29, 2024]
- [5] Netstock, “The ultimate list of inventory management statistics,” Netstock, Apr. 22, 2022. Available: <https://www.netstock.com/blog/the-ultimate-list-of-inventory-management-statistics/#Inventory-management-trends>. [Accessed: May 29, 2024]
- [6] “Smart Shelves Market Size, Share & Growth Analysis Report - 2032,” *MarketsandMarkets*, 2022. Available: <https://www.marketsandmarkets.com/Market-Reports/smart-shelf-market-101938228.html>. [Accessed: May 29, 2024]

13.0 APPENDICES

Data analytics:

```
import streamlit as st
import pandas as pd

# Initial data
if 'data' not in st.session_state:
    st.session_state.data = pd.DataFrame(columns=["Water", "Snack", "Instant Noodle"])

# Function to add values to the table
def add_values():
    new_value1 = st.number_input("Enter value for Water", value=0, key='Water')
    new_value2 = st.number_input("Enter value for Snack", value=0, key='Snack')
    new_value3 = st.number_input("Enter value for Instant Noodle", value=0, key='Instant Noodle')
    if st.button("Add to Table"):
        new_row = pd.DataFrame({"Water": [new_value1], "Snack": [new_value2], "Instant Noodle": [new_value3]})
        st.session_state.data = pd.concat([st.session_state.data, new_row], ignore_index=True)

# Function to plot the line chart
def plot_chart():
    if st.button("Plot Line Chart"):
        st.line_chart(st.session_state.data)

st.title("Analysis Website for total sale")

# Adding values to the table
add_values()

# Display the table
st.write("Table of Values")
st.write(st.session_state.data)

# Plotting the line chart
plot_chart()
```

Website for real-time tracking:

```
import streamlit as st
import pandas as pd
import time
```

```

from datetime import datetime
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import serial

# Ensure pyserial is imported correctly
try:
    import serial
except ImportError:
    st.error("Please install pyserial by running `pip install pyserial`")
    exit()

# Set up the serial connection (adjust the COM port and baud rate as
# necessary)
try:
    ser = serial.Serial('COM9', 9600, timeout=0.1) # Replace 'COM9' with
your Arduino's serial port
    ser1 = serial.Serial('COM10', 9600, timeout=0.1) # Replace 'COM9' with
your Arduino's serial port
    ser2 = serial.Serial('COM11', 9600, timeout=0.1) # Replace 'COM9' with
your Arduino's serial port

except serial.SerialException:
    st.error("Could not open serial port. Please check the port name and try
again.")
    exit()

# Initialize Streamlit app
st.title('StockWise')
st.header('CoopMart Kolej 10')

# Initialize empty DataFrames for each product
product1_df = pd.DataFrame(columns=["Timestamp", "Product Name", "Price",
"Stock", "Action"])
product2_df = pd.DataFrame(columns=["Timestamp", "Product Name", "Price",
"Stock", "Action"])
product3_df = pd.DataFrame(columns=["Timestamp", "Product Name", "Price",
"Stock", "Action"])

# Dictionary to store daily sales
daily_sales = {
    "Water": {"Total Sell": 0, "Total Price": 0.0},
    "Snack": {"Total Sell": 0, "Total Price": 0.0},
    "Instant Noodle": {"Total Sell": 0, "Total Price": 0.0}
}

```



```

# Function to read and parse serial data
def read_serial_data():
    try:
        line = ser.readline().decode('utf-8').strip()

        line1 = ser1.readline().decode('utf-8').strip()

        line2 = ser2.readline().decode('utf-8').strip()

        if line.startswith("Product Name:"):
            name = line.split(": ")[1]
            price = float(ser.readline().decode('utf-8').strip().split(":
RM"))[1])
            availability =
int(ser.readline().decode('utf-8').strip().split(": ")[1])
            action = "Out" # Assuming every scan results in an 'Out' action
            return datetime.now(), name, price, availability, action

        if line1.startswith("Product Name:"):
            name = line1.split(": ")[1]
            price = float(ser1.readline().decode('utf-8').strip().split(":
RM"))[1])
            availability =
int(ser1.readline().decode('utf-8').strip().split(": ")[1])
            action = "Out" # Assuming every scan results in an 'Out' action
            return datetime.now(), name, price, availability, action

        if line2.startswith("Product Name:"):
            name = line2.split(": ")[1]
            price = float(ser2.readline().decode('utf-8').strip().split(":
RM"))[1])
            availability =
int(ser2.readline().decode('utf-8').strip().split(": ")[1])
            action = "Out" # Assuming every scan results in an 'Out' action
            return datetime.now(), name, price, availability, action

    except serial.SerialException as e:
        st.error(f"Serial error: {e}")
    except Exception as e:
        st.error(f"Error reading serial data: {e}")
    return None

# Function to send email alert
def send_email(product_name):
    try:

```

```

# SMTP server configuration
smtp_server = 'smtp.gmail.com'
smtp_port = 587
sender_email = '209997@student.upm.edu.my'
sender_password = 'Zulhisyam4'
receiver_email = '212213@student.upm.edu.my'

# Create a SMTP session
s = smtplib.SMTP(smtp_server, smtp_port)
s.starttls() # Start TLS encryption

# Login to Gmail
s.login(sender_email, sender_password)

# Email content
subject = 'COOP MART INVENTORY MANAGEMENT DATA'

# Customize message based on product_name
if product_name == 'Water':
    message = f"""
    Dear worker,

    This is a reminder to fill up the Stock for Water.

    Best regards,
    Your Manager
    """
elif product_name == 'Snack':
    message = f"""
    Dear worker,

    This is a reminder to fill up the Stock for Snack.

    Best regards,
    Your Manager
    """
elif product_name == 'Instant Noodle':
    message = f"""
    Dear worker,

    This is a reminder to fill up the Stock for Instant Noodle.

    Best regards,
    Your Manager
    """
else:
    # Default message for unknown products
    message = """
    Dear worker,

```

```

        This is a reminder to fill up the Stock for a product.

        Best regards,
        Your Manager
        """

    # Construct the message
    msg = MIMEMultipart()
    msg['From'] = sender_email
    msg['To'] = receiver_email
    msg['Subject'] = subject

    # Attach the message body
    msg.attach(MIMEText(message, 'plain'))

    # Send the email
    s.sendmail(sender_email, receiver_email, msg.as_string())

    # Quit the session
    s.quit()
except Exception as e:
    st.error(f"Error sending email: {e}")

def update_tables():
    global product1_df, product2_df, product3_df

    data = read_serial_data()
    if data:
        timestamp, product_name, price, availability, action = data

        if product_name == "Water":
            # Check if this is the first time availability reaches 0
            if availability == 0 and ("Water" not in product1_df["Product
Name"].values or
                                product1_df.loc[product1_df["Product
Name"] == "Water", "Stock"].iloc[-1] != 0):
                product1_df = pd.concat(
                    [pd.DataFrame([data], columns=["Timestamp", "Product
Name", "Price", "Stock", "Action"]),
                     product1_df], ignore_index=True)
                product1_table.table(product1_df)
            elif product_name == "Snack":
                # Check if this is the first time availability reaches 0
                if availability == 0 and ("Snack" not in product2_df["Product
Name"].values or
                                product2_df.loc[product2_df["Product
Name"] == "Snack", "Stock"].iloc[-1] != 0):

```

```

        product2_df = pd.concat(
            [pd.DataFrame([data], columns=["Timestamp", "Product
Name", "Price", "Stock", "Action"]),
            product2_df], ignore_index=True)
        product2_table.table(product2_df)
    elif product_name == "Instant Noodle":
        # Check if this is the first time availability reaches 0
        if availability == 0 and ("Instant Noodle" not in
product3_df["Product Name"].values or
                                product3_df.loc[product3_df["Product
Name"] == "Instant Noodle", "Stock"].iloc[
                                    -1] != 0):
            product3_df = pd.concat(
                [pd.DataFrame([data], columns=["Timestamp", "Product
Name", "Price", "Stock", "Action"]),
                product3_df], ignore_index=True)
            product3_table.table(product3_df)

    # Update daily sales summary
    update_daily_sales(product_name, 1, price)

    # Send reStock alert if availability is Low
    if availability <= 1:
        send_email(product_name)

    # Update summary table
    summary_data = []
    for product_name, sales_info in daily_sales.items():
        summary_data.append({
            "Product Name": product_name,
            "Total Sell": sales_info["Total Sell"],
            "Total Price": sales_info["Total Price"]
        })
    summary_df = pd.DataFrame(summary_data)
    summary_table.table(summary_df)

# Function to update daily sales summary
def update_daily_sales(product_name, quantity, price):
    global daily_sales
    daily_sales[product_name]["Total Sell"] += quantity
    daily_sales[product_name]["Total Price"] += (quantity * price)

# Create placeholders for displaying product information
product1_placeholder = st.empty()
product2_placeholder = st.empty()
product3_placeholder = st.empty()
summary_placeholder = st.empty() # Placeholder for summary table

```

```

# Set up the layout for the tables
with st.container():
    st.subheader("Product Details")

    with st.expander("Water"):
        product1_table = st.empty()

    with st.expander("Snack"):
        product2_table = st.empty()

    with st.expander("Instant Noodle"):
        product3_table = st.empty()

    st.subheader("Today's Sales Summary")
    summary_table = st.empty()

# Function to update the tables and send email alerts
def update_tables():
    global product1_df, product2_df, product3_df

    data = read_serial_data()
    if data:
        timestamp, product_name, price, availability, action = data
        availability -= 1 # Simulate sale decrementing availability by 1

        if product_name == "Water":
            product1_df = pd.concat(
                [pd.DataFrame([data], columns=["Timestamp", "Product Name",
                "Price", "Stock", "Action"]), product1_df],
                ignore_index=True)
            product1_table.table(product1_df)
        elif product_name == "Snack":
            product2_df = pd.concat(
                [pd.DataFrame([data], columns=["Timestamp", "Product Name",
                "Price", "Stock", "Action"]), product2_df],
                ignore_index=True)
            product2_table.table(product2_df)
        elif product_name == "Instant Noodle":
            product3_df = pd.concat(
                [pd.DataFrame([data], columns=["Timestamp", "Product Name",
                "Price", "Stock", "Action"]), product3_df],
                ignore_index=True)
            product3_table.table(product3_df)

    # Update daily sales summary
    update_daily_sales(product_name, 1, price)

```

```

# Send reStock alert if availability is Low
if availability <= 1:
    send_email(product_name)

# Update summary table
summary_data = []
for product_name, sales_info in daily_sales.items():
    summary_data.append({
        "Product Name": product_name,
        "Total Sell": sales_info["Total Sell"],
        "Total Price": sales_info["Total Price"]
    })
summary_df = pd.DataFrame(summary_data)
summary_table.table(summary_df)

# Main loop to read data from serial and update the transaction log
while True:
    update_tables()
    time.sleep(0.1)

```

First partition:

```

#include <SPI.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
int led1 = 6;
int led2 = 7;

#define RST_PIN 9
#define SS_PIN 10

byte readCard[4];
String MasterTag = "FC7D159F"; // REPLACE this Tag ID with your Tag ID!!!
String tagID = "";

// Product data structure
struct Product {
    String id;
    String name;

```

```

    float price;
    int availability;
};

// Create an array of products
Product products[] = {
    {"FC7D159F", "Water", 2.00, 5}
};

// Create instances
MFRC522 mfrc522(SS_PIN, RST_PIN);

void setup() {
    lcd.begin();
    lcd.backlight();
    SPI.begin(); // SPI bus
    mfrc522.PCD_Init(); // MFRC522
    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);
    lcd.clear();
    lcd.print("Water");
    lcd.setCursor(0, 1);
    lcd.print("RM2.00");
    lcd.print(" ");
    lcd.print("Stock:5");
    Serial.begin(9600); // Initialize serial communication for debugging
}

void loop() {
    // Wait until a new tag is available
    if (getID()) {
        lcd.clear();
        lcd.setCursor(0, 0);
        bool productFound = false;

        // Debugging: Print scanned tag ID

```

```

Serial.print("Scanned Tag ID: ");
Serial.println(tagID);

// Check if the scanned tag matches the product
for (Product &product : products) {
    if (tagID.equals(product.id)) {
        productFound = true;

        // Update availability if greater than 0
        if (product.availability > 0) {
            product.availability--;
        }

        // Display product information along with availability status
        lcd.clear();
        lcd.print(product.name);
        lcd.setCursor(0, 1);
        lcd.print("RM");
        lcd.print(product.price);
        lcd.print(" Stock:");
        lcd.print(product.availability);

        // Print product information to Serial Monitor
        Serial.print("Product Name: ");
        Serial.println(product.name);
        Serial.print("Price: RM");
        Serial.println(product.price);
        Serial.print("Stock: ");
        Serial.println(product.availability);

        // Wait a short time before updating availability status
        delay(500);

        // Display availability status
        lcd.clear();
        if (product.availability == 0) {

```



```

        lcd.print("Out of Stock");
    } else if (product.availability <= 2) {
        lcd.print("Hot-Selling, Please");
        lcd.setCursor(0, 1);
        lcd.print("restock the product");
    } else if (product.availability <= 3) {
        lcd.print("Hot-Selling!");
    } else {
        lcd.print("Not Hot-Selling");
    }

    delay(500);

    // Continue displaying product name, price, availability, and
status
    lcd.clear();
    lcd.print(product.name);
    lcd.setCursor(0, 1);
    lcd.print("RM");
    lcd.print(product.price);
    lcd.print(" Stock:");
    lcd.print(product.availability);
    lcd.setCursor(0, 2);
    if (product.availability == 0) {
        lcd.print("Out of Stock");
    } else if (product.availability <= 2) {
        lcd.print("Hot-Selling, restock");
    } else if (product.availability <= 3) {
        lcd.print("Hot-Selling!");
    } else {
        lcd.print("Not Hot-Selling");
    }

    break;
}
}

```

```

        if (!productFound) {
            lcd.clear();
            lcd.print("Unknown Product");
            Serial.println("Unknown Product");
            delay(500);
        }
    }
}

// Read new tag if available
boolean getID() {
    // Getting ready for Reading PICCs
    if (!mfrc522.PICC_IsNewCardPresent()) { // If a new PICC placed to RFID
reader continue
        return false;
    }
    if (!mfrc522.PICC_ReadCardSerial()) { // Since a PICC placed get Serial
and continue
        return false;
    }
    tagID = "";
    for (uint8_t i = 0; i < 4; i++) { // The MIFARE PICCs that we use have 4
byte UID
        tagID.concat(String(mfrc522.uid.uidByte[i], HEX)); // Adds the 4 bytes
in a single String variable
    }
    tagID.toUpperCase(); // Convert to uppercase
    mfrc522.PICC_HaltA(); // Stop reading
    return true;
}

```

Second partition:

```

#include <SPI.h>
#include <MFRC522.h>

```

```

#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
int led1 = 6;
int led2 = 7;

#define RST_PIN 9
#define SS_PIN 10

byte readCard[4];
String MasterTag = "EC39F12B"; // REPLACE this Tag ID with your Tag ID!!!
String tagID = "";

// Product data structure
struct Product {
    String id;
    String name;
    float price;
    int availability;
};

// Create an array of products
Product products[] = {
    {"EC39F12B", "Snack", 1.00, 5}
};

// Create instances
MFRC522 mfrc522(SS_PIN, RST_PIN);

void setup() {
    lcd.begin();
    lcd.backlight();
    SPI.begin(); // SPI bus
    mfrc522.PCD_Init(); // MFRC522
    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);
}

```

```

    lcd.clear();
    lcd.print("Snack");
    lcd.setCursor(0, 1);
    lcd.print("RM2.00");
    lcd.print("  ");
    lcd.print("Stock:5");
    Serial.begin(9600); // Initialize serial communication for debugging
}

void loop() {
    // Wait until a new tag is available
    if (getID()) {
        lcd.clear();
        lcd.setCursor(0, 0);
        bool productFound = false;

        // Debugging: Print scanned tag ID
        Serial.print("Scanned Tag ID: ");
        Serial.println(tagID);

        // Check if the scanned tag matches the product
        for (Product &product : products) {
            if (tagID.equals(product.id)) {
                productFound = true;

                // Update availability if greater than 0
                if (product.availability > 0) {
                    product.availability--;
                }

                // Display product information along with availability status
                lcd.clear();
                lcd.print(product.name);
                lcd.setCursor(0, 1);
                lcd.print("RM");
                lcd.print(product.price);
            }
        }
    }
}

```

```

    lcd.print(" Stock:");
    lcd.print(product.availability);

    // Print product information to Serial Monitor
    Serial.print("Product Name: ");
    Serial.println(product.name);
    Serial.print("Price: RM");
    Serial.println(product.price);
    Serial.print("Stock: ");
    Serial.println(product.availability);

    // Wait a short time before updating availability status
    delay(500);

    // Display availability status
    lcd.clear();
    if (product.availability == 0) {
        lcd.print("Out of Stock");
    } else if (product.availability <= 2) {
        lcd.print("Hot-Selling, Please");
        lcd.setCursor(0, 1);
        lcd.print("restock the product");
    } else if (product.availability <= 3) {
        lcd.print("Hot-Selling!");
    } else {
        lcd.print("Not Hot-Selling");
    }

    delay(500);

    // Continue displaying product name, price, availability, and
status
    lcd.clear();
    lcd.print(product.name);
    lcd.setCursor(0, 1);
    lcd.print("RM");

```

```

        lcd.print(product.price);
        lcd.print(" Stock:");
        lcd.print(product.availability);
        lcd.setCursor(0, 2);
        if (product.availability == 0) {
            lcd.print("Out of Stock");
        } else if (product.availability <= 2) {
            lcd.print("Hot-Selling, restock");
        } else if (product.availability <= 3) {
            lcd.print("Hot-Selling!");
        } else {
            lcd.print("Not Hot-Selling");
        }

        break;
    }
}

if (!productFound) {
    lcd.clear();
    lcd.print("Unknown Product");
    Serial.println("Unknown Product");
    delay(500);
}
}
}

// Read new tag if available
boolean getID() {
    // Getting ready for Reading PICCs
    if (!mfrc522.PICC_IsNewCardPresent()) { // If a new PICC placed to RFID
reader continue
        return false;
    }
    if (!mfrc522.PICC_ReadCardSerial()) { // Since a PICC placed get Serial
and continue

```

```

        return false;
    }
    tagID = "";
    for (uint8_t i = 0; i < 4; i++) { // The MIFARE PICCs that we use have 4
byte UID
        tagID.concat(String(mfrc522.uid.uidByte[i], HEX)); // Adds the 4 bytes
in a single String variable
    }
    tagID.toUpperCase(); // Convert to uppercase
    mfrc522.PICC_HaltA(); // Stop reading
    return true;
}

```

Third partition:

```

#include <SPI.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
int led1 = 6;
int led2 = 7;

#define RST_PIN 9
#define SS_PIN 10

byte readCard[4];
String MasterTag = "EE45159F"; // REPLACE this Tag ID with your Tag ID!!!
String tagID = "";

// Product data structure
struct Product {
    String id;
    String name;
    float price;
    int availability;
}

```

```

};

// Create an array of products
Product products[] = {
    {"EE45159F", "Instant Noodle", 5.00, 5}
};

// Create instances
MFRC522 mfrc522(SS_PIN, RST_PIN);

void setup() {
    lcd.begin();
    lcd.backlight();
    SPI.begin(); // SPI bus
    mfrc522.PCD_Init(); // MFRC522
    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);
    lcd.clear();
    lcd.print("Instant Noodle");
    lcd.setCursor(0, 1);
    lcd.print("RM5.00");
    lcd.print(" ");
    lcd.print("Stock:5");
    Serial.begin(9600); // Initialize serial communication for debugging
}

void loop() {
    // Wait until a new tag is available
    if (getID()) {
        lcd.clear();
        lcd.setCursor(0, 0);
        bool productFound = false;

        // Debugging: Print scanned tag ID
        Serial.print("Scanned Tag ID: ");
        Serial.println(tagID);
    }
}

```



```

// Check if the scanned tag matches the product
for (Product &product : products) {
    if (tagID.equals(product.id)) {
        productFound = true;

        // Update availability if greater than 0
        if (product.availability > 0) {
            product.availability--;
        }

        // Display product information along with availability status
        lcd.clear();
        lcd.print(product.name);
        lcd.setCursor(0, 1);
        lcd.print("RM");
        lcd.print(product.price);
        lcd.print(" Stock:");
        lcd.print(product.availability);

        // Print product information to Serial Monitor
        Serial.print("Product Name: ");
        Serial.println(product.name);
        Serial.print("Price: RM");
        Serial.println(product.price);
        Serial.print("Stock: ");
        Serial.println(product.availability);

        // Wait a short time before updating availability status
        delay(500);

        // Display availability status
        lcd.clear();
        if (product.availability == 0) {
            lcd.print("Out of Stock");
        } else if (product.availability <= 2) {

```

```

        lcd.print("Hot-Selling, Please");
        lcd.setCursor(0, 1);
        lcd.print("restock the product");
    } else if (product.availability <= 3) {
        lcd.print("Hot-Selling!");
    } else {
        lcd.print("Not Hot-Selling");
    }

    delay(500);

    // Continue displaying product name, price, availability, and
status
    lcd.clear();
    lcd.print(product.name);
    lcd.setCursor(0, 1);
    lcd.print("RM");
    lcd.print(product.price);
    lcd.print(" Stock:");
    lcd.print(product.availability);
    lcd.setCursor(0, 2);
    if (product.availability == 0) {
        lcd.print("Out of Stock");
    } else if (product.availability <= 2) {
        lcd.print("Hot-Selling, restock");
    } else if (product.availability <= 3) {
        lcd.print("Hot-Selling!");
    } else {
        lcd.print("Not Hot-Selling");
    }

    break;
}
}

if (!productFound) {

```

```

        lcd.clear();
        lcd.print("Unknown Product");
        Serial.println("Unknown Product");
        delay(500);
    }
}

// Read new tag if available
boolean getID() {
    // Getting ready for Reading PICCs
    if (!mfrc522.PICC_IsNewCardPresent()) { // If a new PICC placed to RFID
reader continue
        return false;
    }
    if (!mfrc522.PICC_ReadCardSerial()) { // Since a PICC placed get Serial
and continue
        return false;
    }
    tagID = "";
    for (uint8_t i = 0; i < 4; i++) { // The MIFARE PICCs that we use have 4
byte UID
        tagID.concat(String(mfrc522.uid.uidByte[i], HEX)); // Adds the 4 bytes
in a single String variable
    }
    tagID.toUpperCase(); // Convert to uppercase
    mfrc522.PICC_HaltA(); // Stop reading
    return true;
}

```